

COMPILADORES

Diseño e Implementación de un *Compilador Fuente-a-Fuente*

Propuesta de un lenguaje de programación de origen con palabras clave en español, análisis léxico sobre dicha gramática e implementación en Python de un tokenizador interactivo.

MODALIDAD	LENGUAJE ORIGEN	LENGUAJE DESTINO
Equipos de trabajo	A construir	Preseleccionado

Análisis léxico — Gramática BNF — Python Tokenizer

Documento creado por Claude IA y revisado, corregido y aumentado por: Carlos Andrés Arbeláez Velásquez

§ 1 — CONTEXTO Y MOTIVACIÓN

Descripción general del proyecto

En el marco del curso de Teoría de Compiladores, cada equipo de trabajo ha seleccionado previamente un lenguaje de programación de destino (p. ej. Python, JavaScript, Java, C, entre otros). El proyecto consiste en diseñar un lenguaje de programación propio —denominado lenguaje fuente— con palabras reservadas tomadas del español, y en construir, a lo largo de las entregas del curso, un compilador completo que traduzca programas escritos en ese lenguaje fuente al lenguaje de destino ya seleccionado.

Esta primera entrega cubre dos productos independientes: (a) la especificación formal del lenguaje fuente mediante una Gramática Libre de Contexto en notación BNF, y (b) una aplicación funcional que realice el análisis léxico de cadenas de texto escritas conforme a dicha gramática.

PRINCIPIO ORIENTADOR

Antes de proponer el lenguaje fuente, el equipo debe analizar el lenguaje de destino ya seleccionado e identificar sus características fundamentales: paradigma (imperativo, orientado a objetos, funcional), tipado, estructuras de control disponibles, mecanismo de definición de subprogramas (funciones, métodos, clases), entre otros. El lenguaje fuente propuesto debe ser coherente con el lenguaje de destino en términos de expresividad, de modo que la compilación sea factible y los constructos de la fuente tengan una traducción natural al destino.

§ 2 — LENGUAJE FUENTE PROPUESTO

Requisitos del lenguaje de origen

El lenguaje fuente debe cumplir los siguientes requisitos mínimos de expresividad. No se trata de construir un lenguaje de juguete, sino que debe ser capaz de expresar algoritmos reales de mediana complejidad.

Capacidades mínimas exigidas

1. Palabras reservadas e identificadores escritos en español (incluyendo tildes y la letra ñ como caracteres válidos en identificadores si llega a ser necesario).
2. Operaciones aritméticas: suma, resta, multiplicación, división, módulo y potencia.
3. Operaciones lógicas: conjunción (y), disyunción (o), negación (no).
4. Operaciones relacionales: igual, distinto, menor, mayor, menor o igual, mayor o igual.
5. Estructura condicional si ... entonces ... sino ... fin_si (equivalente a if-else).
6. Estructura de repetición para ... hacer ... fin_para con variable de control (equivalente a for).
7. Estructura de repetición mientras ... hacer ... fin_mientras (equivalente a while).

8. Definición de funciones o clases/objetos según corresponda al paradigma del lenguaje de destino.
9. Tipos de datos básicos: enteros, reales, cadenas de texto y booleanos, con palabras clave en español.
10. Instrucción de asignación e instrucción de retorno (retornar).

§ 2.1 — Funciones, clases u objetos según el lenguaje de destino

La elección entre definir funciones o clases y objetos en el lenguaje fuente depende del paradigma del lenguaje de destino seleccionado por el equipo:

EQUIPOS CON DESTINO ORIENTADO A OBJETOS (JAVA, C#, ETC.)

El lenguaje fuente debe incluir mecanismos para definir clases con atributos y métodos, operador de instanciación y acceso a miembros. Las palabras reservadas deberán ser en español (p. ej. clase, nuevo, este).

EQUIPOS CON DESTINO PROCEDURAL O DE SCRIPTING (PYTHON, JAVASCRIPT, C, ETC.)

Es suficiente con definir funciones con parámetros formales, cuerpo y valor de retorno. Se recomienda también soportar funciones anidadas si el lenguaje de destino las soporta. La palabra reservada puede ser funcion o procedimiento.

§ 3 — ESPECIFICACIÓN FORMAL

Gramática Libre de Contexto en BNF

El primer entregable teórico es la Gramática Libre de Contexto (GLC) que describe completamente el lenguaje fuente propuesto. La gramática debe presentarse en formato BNF estándar y satisfacer las siguientes restricciones técnicas:

- Sin recursividad por la izquierda: ninguna producción de la forma $A \rightarrow A\alpha$ (directa o indirecta) puede aparecer en la gramática.
- Sin ambigüedad en los prefijos comunes: cuando un no-terminal tiene múltiples alternativas que comienzan con el mismo símbolo, se debe aplicar factorización por la izquierda.
- Analizador descendente recursivo: la gramática debe poder analizarse con un analizador LL(1) o LL(k) con k pequeño; esto implica que los conjuntos PRIMERO de cada alternativa sean disjuntos.
- La gramática debe cubrir todos los constructos listados en §2, sin excepción.

ESTRUCTURA SUGERIDA PARA EL DOCUMENTO DE GRAMÁTICA

El documento debe comenzar con la descripción informal del lenguaje (vocabulario, tipos, alcance), seguir con el conjunto de terminales agrupados por categoría, luego las producciones BNF

ordenadas de arriba (programa) hacia abajo (expresiones atómicas), y finalizar con ejemplos de programas válidos e inválidos.

Ejemplo parcial de gramática BNF orientativa

A continuación se muestra un fragmento ilustrativo y no normativo de cómo podría lucir parte de la gramática. Cada equipo debe proponer su propia versión completa y consistente.

```
/* Programa y declaraciones */
<programa>          ::= <declaracion>*
<declaracion>       ::= <def_funcion>
                       | <def_clase>
                       | <sentencia>

/* Definicion de funcion */
<def_funcion>       ::= funcion <IDENTIFICADOR>
                       "(" <parametros> ")"
                       hacer <bloque> fin_funcion
<parametros>        ::= <param_lista> | e
<param_lista>       ::= <tipo> <IDENTIFICADOR> <param_lista'>
<param_lista'>      ::= "," <tipo> <IDENTIFICADOR> <param_lista'> | e

/* Sentencias */
<bloque>            ::= <sentencia>+
<sentencia>         ::= <asignacion> | <sent_si>
                       | <sent_para> | <sent_mientras>
                       | <sent_retornar> | <llamada_funcion>

/* If-Else (sin recursividad izquierda) */
<sent_si>           ::= si <expresion> entonces
                       <bloque>
                       <rama_sino>
                       fin_si
<rama_sino>         ::= sino <bloque> | e

/* For */
<sent_para>         ::= para <IDENTIFICADOR> desde <expresion>
                       hasta <expresion> <paso_opt>
                       hacer <bloque> fin_para
<paso_opt>          ::= paso <expresion> | e

/* While */
<sent_mientras>     ::= mientras <expresion>
                       hacer <bloque> fin_mientras

/* Expresiones (eliminando recursividad izquierda) */
<expresion>         ::= <expr_or>
<expr_or>           ::= <expr_and> <expr_or'>
<expr_or'>          ::= o <expr_and> <expr_or'> | e
```

```

<expr_and>      ::= <expr_rel> <expr_and'>
<expr_and'>     ::= y <expr_rel> <expr_and'> | e
<expr_rel>      ::= <expr_add> <expr_rel'>
<expr_rel'>     ::= <op_rel> <expr_add> | e
<op_rel>        ::= "==" | "!=" | "<" | ">" | "<=" | ">="
<expr_add>      ::= <expr_mul> <expr_add'>
<expr_add'>     ::= "+" <expr_mul> <expr_add'>
                | "-" <expr_mul> <expr_add'>
                | e
<expr_mul>      ::= <expr_unaria> <expr_mul'>
<expr_mul'>     ::= "*" <expr_unaria> <expr_mul'>
                | "/" <expr_unaria> <expr_mul'>
                | "%" <expr_unaria> <expr_mul'>
                | e
<expr_unaria>   ::= no <expr_unaria>
                | "-" <expr_unaria>
                | <expr_primaria>
<expr_primaria> ::= <NUMERO_ENTERO> | <NUMERO_REAL>
                | <CADENA>
                | verdadero | falso
                | <IDENTIFICADOR> <sufijo_id>
                | "(" <expresion> ")"
<sufijo_id>     ::= "(" <argumentos> ")" /* llamada */
                | e

/* Tipos */
<tipo>         ::= entero | real | cadena | booleano

```

ATENCIÓN — LA GRAMÁTICA ANTERIOR ES SOLO ORIENTATIVA

Cada equipo debe proponer y defender su propia gramática completa. El fragmento de arriba ilustra el estilo BNF esperado, con eliminación de recursividad izquierda y la jerarquía de precedencia de operadores. La gramática entregada debe cubrir la totalidad del lenguaje, incluyendo declaraciones de variables, definición de clases (si aplica) y el manejo de comentarios.

§ 4 — ENTREGABLE PRÁCTICO

Analizador léxico

El segundo entregable es una aplicación que implemente el analizador léxico (lexer o scanner) del lenguaje fuente propuesto. Esta aplicación sentará las bases para el analizador sintáctico que se desarrollará en entregas posteriores.

Requerimientos funcionales

11. Entrada de cadena. El programa debe ofrecer al usuario dos modos de ingreso: (a) digitación libre de una cadena de texto directamente en la consola o interfaz, y (b) selección de una cadena desde una lista de cadenas predefinidas que cubran distintos programas del lenguaje.
12. Visualización gráfica de tokens. El programa debe mostrar de forma visual cómo la cadena de entrada se divide en tokens, destacando cada token con su categoría (TokenType). Se recomienda el uso de colores en consola.
13. Tabla de símbolos léxicos. Se debe presentar una tabla que liste por cada token: el lexema (texto exacto), la categoría (TokenType), la fila (línea) y la columna del texto de entrada donde fue encontrado.
14. Manejo de errores léxicos. Cuando la entrada contenga un carácter o secuencia que no corresponda a ningún token válido, el programa debe reportar un error léxico indicando la posición (fila y columna), sin abortar el análisis del resto de la cadena.
15. Independencia del analizador léxico. La lógica del lexer debe estar encapsulada en una clase o módulo independiente, de modo que pueda reutilizarse sin modificaciones en el analizador sintáctico de entregas futuras.

§ 5 — ENTREGABLES

Resumen de entregables

Entregable 1 — Teórico: Gramática BNF

- Descripción informal del lenguaje
- Conjunto de tokens terminales
- Producciones BNF (sin recursividad izquierda)
- Verificación de condición LL
- Ejemplos de programas válidos
- Justificación de coherencia con el lenguaje destino

Entregable 2 — Práctico: Analizador léxico Python

- Módulo `lexer.py` reutilizable
- Ingreso de cadena libre o predefinida
- Visualización gráfica de tokens
- Tabla de símbolos con fila y columna
- Reporte de errores léxicos
- Código documentado + README

NOTA SOBRE LAS ENTREGAS FUTURAS

En entregas posteriores, el equipo construirá sobre este trabajo el analizador sintáctico descendente recursivo (que consumirá los tokens producidos por el lexer de esta entrega), seguido del análisis semántico, y finalmente la traducción al lenguaje de destino seleccionado.

§ 6 — EVALUACIÓN

Criterios de evaluación

Criterio	Peso	Descripción
Gramática BNF — completitud	20%	Cubre todos los constructos exigidos (operaciones, estructuras de control, funciones/clases).
Lexer — corrección funcional	10%	Reconoce correctamente todos los tokens definidos en la gramática.
Visualización de tokens	10%	Representación gráfica clara, con colores por categoría y cadenas predefinidas funcionales.
Tabla de símbolos	10%	Lexema, tipo, fila y columna correctamente registrados para cada token.
Manejo de errores léxicos	10%	Detección y reporte de caracteres inválidos sin abortar el análisis.
Sustentación (exposición)	40%	Presentar coherentemente los productos presentados, incluyendo explicación sobre cómo se usó la I.A. para lograrlos. También incluye la habilidad para responder preguntas sobre los productos, de modo que se pueda dar cuenta que se se comprenden los productos que son entregados.

Este documento establece los requisitos mínimos. El equipo puede extender el lenguaje propuesto siempre que cumpla con todas las restricciones aquí indicadas.